1 Write Amp

SSD erase units cover multiple pages. Outright in-place updating is expensive since erase granularity is course, so the SSD just marks pages as invalid instead. WA is the phenomenon whereas amt of physical space written is larger than the logical amount the application writes. Over-provisioning is the practice of setting the logical storage capacity lower than the physical capacity, effectively increasing the proportion of invalid pages. This makes it easier for the GC to find erase units with little live data left. Best for all data in an erase unit to be rendered useless at the same time.

- Logical address space L
- Physical capacity P
- Usable capacity $= \frac{L}{P} = 1$

Pages in an SSD can be:

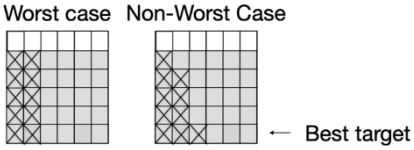
- Empty – writable
- Invalid – to be GC'd
- Valid – contains in-use data

How do states relate to each other:

$$\frac{L}{P} = \frac{\text{valid}}{\text{empty} + \text{valid} + \text{invalid}}$$

1.1 WC Write Amp

Occurs: when invalid pages are uniform across erase units.



Best erase unit to erase is the one with the most invalid pages.

SSD WA approximation: where x is % valid pages:

$$1 + \left(\frac{x}{1-x}\right)$$

WA Cost model assuming uniformly randomly distributed writes: where $\frac{L}{P} = \frac{\text{valid}}{\text{empty+valid+invalid}}$

$$1 + \frac{1}{2} \cdot \frac{L/P}{1 - L/P}$$

This gets multiplied by B if you are writing data that is too small.

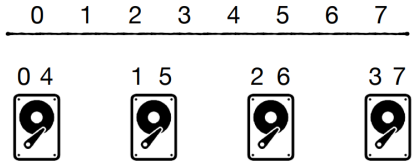
1.2 Making the Most of your SSD

1. No small updates, updates take at least 4KB
2. Avoid random updates, since $\text{GC}_{\text{WA}} = 1 + \frac{\text{migrated}}{\text{freed}} = 1 + \frac{x}{1-x}$
3. Prioritize R over W
4. R & W should be page aligned
5. Use async I/Os

2 RAID

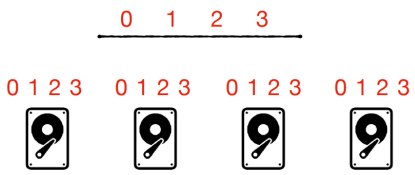
Virtualizes many drives at once to speed things up and provide fail-safes

2.1 Raid O



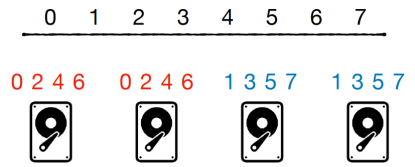
1. Seq R/W at combined bandwidth of all drives
2. Faster random reads/writes due to load balancing
3. No failure tolerance due to no redundancy

2.2 Raid 1



1. Writes as fast as bandwidth of single drive
2. Reads in combined bandwidth of all drives
3. Costs a lot of storage

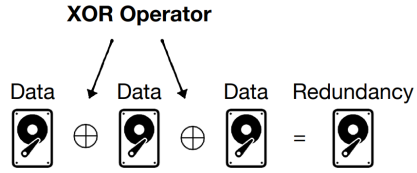
2.3 Raid 0+1



Stripe and mirror at the same time N drives, X drives per mirror group. Then:

1. Sequential writes at $\frac{N}{X}$ bandwidth
2. Reads at combined bandwidth of N
3. Capacity: $\frac{1}{X}$ of total

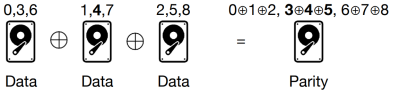
2.4 Raid 4



Problems: performance bottleneck by the single drive of redundancy

Updating a whole stripe is easy, updating one page costs 2 reads and 2 writes

$$4_{\text{new}} \oplus 4_{\text{old}} \oplus (3 \oplus 4 \oplus 5_{\text{old}})$$
$$= 3 \oplus 4 \oplus 5_{\text{new}}$$



2.5 Raid 5

Raid 5 combines striping and distributed parity where redundancy is shared across all drives.

Random writes require 2 reads, 2 writes. Sequential writes at $\frac{N-1}{N}$ of max bandwidth.

3 Buffer Pools

The idea of buffers: keep hot pages in memory. Accessing the same pages over and over, given no writes in-between, are free

Why storing sequentially is better

Reading/writing from storage of units less than 4KB does not pay off:

- Disks: needles must move. For reading pages sequentially that stops becoming a problem
- SSDs: an entire page gets read or not at all

3.1 Tables

A table has:

- N entries
- B entries per DB page
- Whole table fits in $\frac{N}{B}$ pages

3.2 Continuous Scans

- Mixing table rows everywhere has WC $O(N)$ I/O reads
- Storing table rows contiguously has **WC** $O\left(\frac{N}{B}\right)$ I/O reads

3.3 Storing Pages

- Storing pages as a linked list, where previous pages point to the next requires synchronous I/Os, preventing SSD parallelism.
- Directories, each pointing to many singular pages does not saturate a disk's sequential bandwidth since each directory entry only points to one page
- Directories with extents (contiguous 8-64 pages) combine the best of both worlds and can grow into a tree

3.4 What each table stores

- Data type
- Size

- Start address

Databases keep track of free pages/extents by using a bitmap.

3.5 Supporting Indexless Deletes and Updates

Scan table, create holes for deletes and update in place for updates, $O(1)$ writes and $O\left(\frac{N}{B}\right)$ reads due to the need to read everything

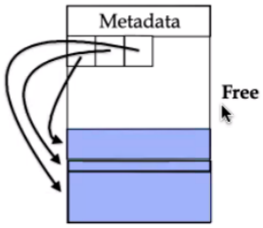
3.6 Optimizing Insertions

- **Scanning and looking** for free space costs $O\left(\frac{N}{B}\right)$ reads and $O(1)$ writes.
- **Storing extents and knowing which are full and which are not** reduces the read time to $O(1)$, cost of accessing directory table. Note: does not make variable-sized entry access faster
- **Buffering insertions** eliminates the cost of reads and reduces write cost to $O\left(\frac{1}{B}\right)$ (every B writes perform a flush). Duplicates are allowed so no overwriting occurs.

3.7 Internal Page Organization

1. Similar to a Python list (deletes take forever)
2. Bitmap (no reorganization but requires more space)

3.7.1 Variable-Sized Row Organization



Using pointers instead of delimiters allow for random access, which is faster

4 Indexing Options

Non-B-Tree versions of indexing. This is where you might want to quickly know the page number given indexable item.

4.1 Zone Maps

Each page stores the min and max. No, this doesn't make it any faster – WC $O(N/B)$ I/Os

4.2 Sort The Column

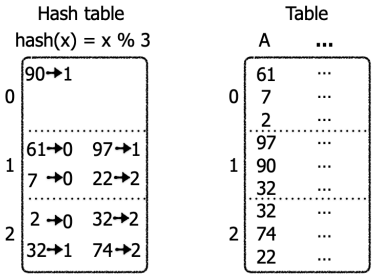
Can only sort one column at once, search costs $O\left(\log_2\left(\frac{N}{B}\right)\right)$ I/O for a total CPU cost of $O\left(\log_2\left(\frac{N}{B}\right) + \log_2(B)\right)$, but updates cost $O\left(\frac{N}{B}\right)$.

4.3 Binary Tree

I/O **and** CPU cost for reads and writes are $O(\log_2(N))$. They don't exploit page locality.

4.4 Hash Table

Give up on sortedness and scan query optimization for average case $O(1)$. Map entries to page. Key repetitions are not allowed.



Downsides:

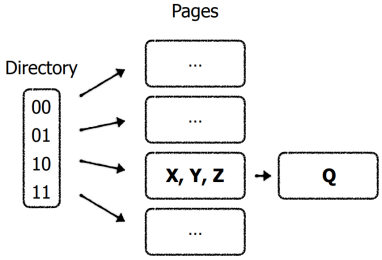
- Bad space efficiency after expansion (50%)
- Only == comparisons
- Expansion leads to performance slumps

A fix for that?

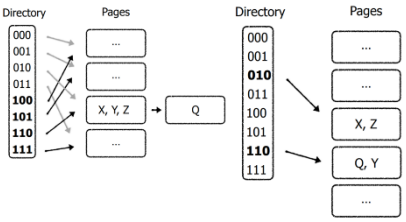
4.5 Extendible Hashing

A directory maps pages in storage with a given hash suffix.

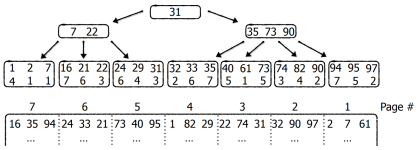
Overflows are handled through chaining.



On capacity, double directory size. **New directory slots still point to previous pages.** Now only one overflowing bucket needs to be expanded at a time.

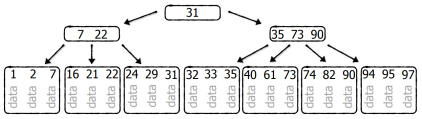


5 B-Tree



Where B is how many entries fit in a page:

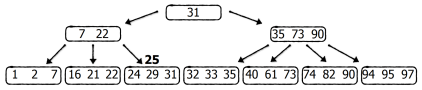
- **Point search** costs $O(\log_B N)$, the height of the tree, since that's the pages it needs to search
  - $O(\log_2(N))$ CPU
- **Writes** cost $O(\log_B N)$ reads and $O(1)$ writes. $O(1)$ reads if internal nodes are stored in memory, where $O\left(\frac{N}{B}\right)$ memory is used for that
- **Scans** cost $O(\log_B N + S)$ reads. If the index is **clustered**, that is table data is stored within the index, that reduces to $O\left(\log_B(N) + \frac{S}{B}\right)$



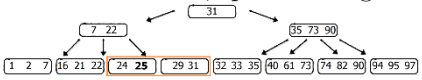
5.1 Going over Inserts (No Propagation)

How to insert?

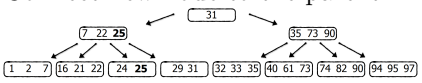
First, find the target node



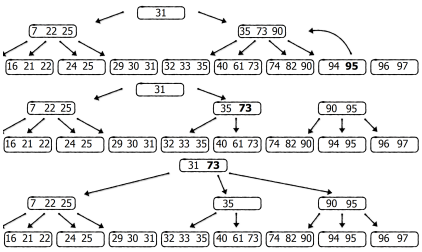
If the node is full, split the target node



Connect new node to the parent



5.2 Inserts With Propagation



The read I/Os per insertion (assuming no storage in memory) remains $O(\log_B N)$, as we need to locate where to insert.

For writes:

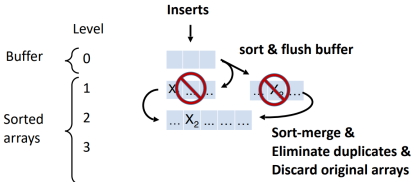
- Every insertion updates 1 leaf
- One in $O\left(B^{-1}\right)$ insertions triggers leaf split

- One in $O\left(B^{-2}\right)$ insertions triggers parent split
- One in $O\left(B^{-3}\right)$ insertions triggers grandparent split

5.3 Why Can We Just Store Internal Nodes In Memory?

Moreover, why do we usually assume that internal nodes are in the buffer pool? Because we access them so often. If a B-tree has $O(N)$ keys, $O\left(N/B\right)$ of them are stored in memory. In practice, this happens all of the time, and there are fewer internal nodes than there are keys in total.

6 LSM Trees



Stored as a B-tree, each sorted array is
Deletes: insert a tombstone entry (the tombstone is a **value**).
Variables:

- $P =$ no. entries that fit in the buffer (level 0). We assume LSM trees are clustered by default, so the size of an entry directly impacts P if memory remains constant

Point searches cost

$$O\left(\underbrace{\log_2\left(\frac{N}{P}\right)}_{L} \cdot \underbrace{\log_B(N)}_{\text{B-tree search}}\right)$$

Since:

- $O\left(\log_2\left(\frac{N}{P}\right)\right)$ levels to search
- $O\left(\log_B(N)\right)$ cost of searching level (if it's organized in a B-tree) (only one sorted run per level)

6.1 LSM Tree Scans

Return most recent version of each entry in the range across the entire tree.
Scans cost:

$$O\left(\log_2\left(\frac{N}{P}\right)\log_B(N) + \frac{S}{B}\right)$$

From this point onwards, we will assume LSM trees store internal nodes for each B-tree in memory.

6.2 LSM Analysis Internal Nodes Memory

Queries (point)

$$\text{Searches} = O(L)$$

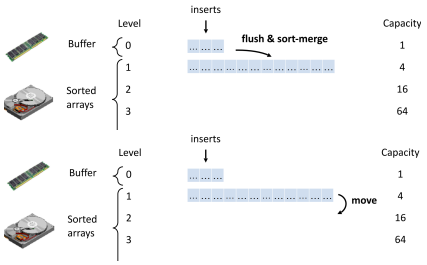
$$\text{Insertions} = O\left(\frac{L}{B}\right)$$

$$L = O\left(\log_2\left(\frac{N}{P}\right)\right)$$

Rationale: searches must go through each level. Insertions may result in a total of L write amplifications since each element can be merged a total of L times.

6.3 Leveled LSM Trees

Improves read performance at the expense of writes
Reduce number of levels by increasing size ratio T
Level count changes to $O\left(\log_T\left(\frac{N}{P}\right)\right)$
Example: size ratio of 4:



Why does this slow down writes?
Because elements get sort-merged more often. In the worst case, each element is re-written T times per level. If every element makes it to the bottom, inserts have a WA of $T \cdot L$.
Changes in cost:

$$\text{Lookup} = O(L)$$

$$\text{Insertion} = O\left(\frac{\mathbf{T}L}{B}\right)$$

$$L = O\left(\log_T\left(\frac{N}{P}\right)\right)$$

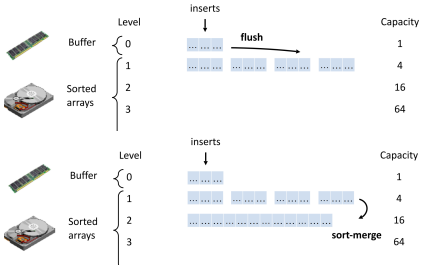
Rationale:

- Less levels than the basic model
- Since only one sorted run per level, and **level count reduced**, looks up are faster
- Writing is more expensive by a factor of T

In the extreme case, where $T = \frac{N}{P}$ (or at most one level), **LSM tree becomes a sorted file**

6.4 Tiered LSM Trees

Improve write performance at the expense of reads
Reduce number of levels by increasing the size ratio T
Example: size ratio of 4



Why does this slow down reads? Because now I need to perform multiple searches per level, because there are more runs.
Changes in cost:

$$\text{Lookup} = O\left(\mathbf{T}L\right)$$

$$\text{Insertion} = O\left(\frac{L}{B}\right)$$

$$L = O\left(\log_T\left(\frac{N}{P}\right)\right)$$

Increasing size ratio to $\frac{N}{P}$ turns the LSM tree into an append-only file

6.5 Conclusions I

LSM trees are:

- Write optimized
- Highly tunable
- Backbone of many modern systems
- Trade-off between lookup and insertion costs

6.6 Runtime Table

All internal nodes are stored in memory.

$$L = \log_T\left(\frac{N}{P}\right)$$

Basic: $T = 2$

\$	Apd. only	Sorted list	B-tree
Q	N/B	$\log\left(\frac{N}{B}\right)$	1
IR	0	N/B	1
IW	$1/B$	N/B	$1 + GC$
M	B		N/B

\$	LSM-BSC	LSM-TRD	LSM-LVL
Q	$\lg\left(\frac{N}{P}\right)$	LT	L
IR	$\frac{\lg\left(\frac{N}{P}\right)}{B}$	$\frac{L}{B}$	$\frac{LT}{B}$
IW	$\frac{\lg\left(\frac{N}{P}\right)}{B}$	$\frac{L}{B}$	$\frac{LT}{B}$
M	N/B	N/B	N/B

Notice how the write costs correspond to write amplification. The write costs correspond to the no. of times an element is copied in the worst case (if we factor out the B).

7 Bloom Filters

Bloom filters are probabilistic sets with the possible operations:

- Add

- Check for inclusion, where:
  - Negative returned for sure, or
  - Positive with error rate (FPR) of ε

It consists of k different hash functions, where its total size can be tweaked with a knob called "bits per entry": M. With that, the total number of bits the filter takes up is $M \times N$.

7.1 Deciding no. of Hash Functions (K)

More memory ⇒ lower FPR
No. of hash functions:

- 1 too low, FPR occurs every collision
- Adding more hash functions exponentially decreases the FPR
- Too many increases it again
- There's an optimal count depending on the bits per entry
  $M = \frac{m \text{ (total bits)}}{N}$

$$k^* = \ln(2)M$$
$$\Rightarrow \varepsilon = 2^{-M\ln(2)}$$

7.2 Construction Contract

If we know:

- N entries to insert
- ε = desired false positive rate (FPR)

Then a filter should be allocated with $N \ln(2) \log_2\left(\frac{1}{\varepsilon}\right)$ bits.

7.3 Bloom Filter Access Cost

$$\text{Insertions} = M\ln(2) = k = O(M)$$
$$\text{Positive query} = M\ln(2) = O(M)$$
$$\text{Avg. Negative query} = 2$$
$$\text{False Positive Rate} = 2^{-M\ln(2)}$$

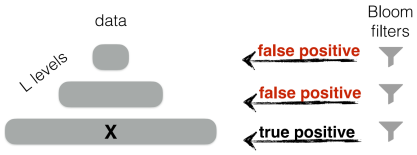
Rationale:

- Insertions: the number of hash functions that must run.
- Positive query: observed all the target bits were 1.
- Avg. negative query: observed one of the target bits was 0 and short circuited.

Note that "positive query" means querying something that's there, in this context.

7.4 Using Bloom Filters with LSM Trees

For a non-tiered LSM tree with L levels, each run of data needs a bloom filter. After all, it's meant to stop a potential I/O read to that level.



7.4.1 Filter Overhead for Basic Accesses

I want to perform a **point query.** The absolute worst case that could happen is **I search for an element that exists, it's on the bottom of the tree, and every bloom filter returns a positive.** Which means $L-1$ false positives and 1 true positive, for a total **worst-case** cost of

$$\underbrace{(L-1)}_{\text{no. FPs}}M + M =$$
$$\text{Worst case} = O(ML)$$

In the average case, false positives usually do not occur so the **average worst case** is:

$$2(L-1) + M$$
$$\text{Avg. worst case} = O(M+L)$$

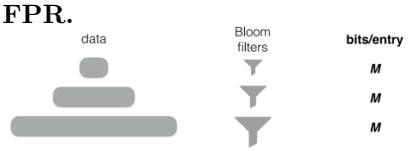
7.5 Optimizing Leveling

Let's optimize a leveled LSM. With bloom filters:

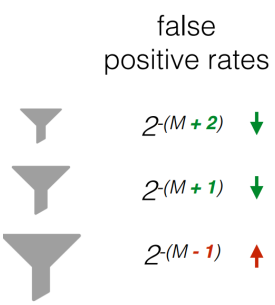
- Gets cost $O\left(2^{-M} \cdot L\right)$
  - Want $O\left(2^{-M}\right)$
  - *MONKEY* stores this
- Inserts cost $O\left(\frac{TL}{B}\right)$
  - Want $O\left(\frac{T+L}{B}\right)$
  - *DOSTOVESKY* solves this

7.5.1 MONKEY (Get optimizer)

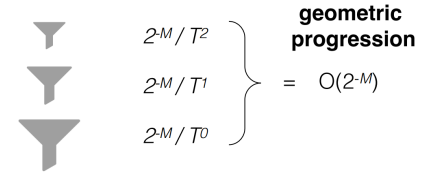
MONKEY OPTIMAL NAVIGABLE KEY-VALUE STORE
Bloom filters scale up the lower the level, **yet each bloom filter has the same FPR.**



If we're accessing our data on the bottom of the LSM, we want our bloom filters higher up to return true negatives, because the data higher is smaller, and we wouldn't want to waste a read. Hence, we can relocate bloom filter sizes, **considering the bottom one is the largest**:



This shrinks the bloom filter on the lowest level and expands the bloom filters above it.



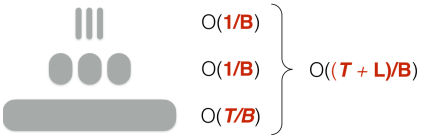
This results in a faster worst-case, $O\left(2^{-M}\right)$ instead of $O\left(2^{-M} \cdot L\right)$. Of course this increases the absolute worst case up to $O\left(ML + L^2\right)$ but that never happens.

7.5.2 DOSTOVESKY (Write Optimizer)

DOSTOEVSKY SPACE TIME OPTIMIZED EVOLVABLE SCALABLE KEY-VALUE STORE
We want to fix the cost of writes being proportional to the number of levels there are. Can be done by making writes lazy on the upper levels. Therefore:

- Apply tiering on the upper levels
- Apply leveling on the lowest level

This reduces the insert I/O cost from $O\left(\frac{TL}{B}\right)$ to $O\left(\frac{T+L}{B}\right)$.



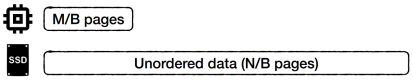
8 External Sorting

Mergesort is worst $O(n\log(n))$ and takes 2x more space. Quicksort has average worst case $O(n\log(n))$ and is done in-place. These only work in memory. So, **what is the optimal way to sort lots of data in storage?**
Here are some attempts:

- B-tree
  - Scan unordered data, insert each into B-tree, append afterwards for $O(N\log_B N)$ R and $O(N)$ W
  - If internal nodes in mem: $O(N)$ R and W
- LSM-tree

– $O\left(\frac{N}{B}\log_2\left(\frac{N}{P}\right)\right)$ reads and writes all the time

How can we efficiently sort data that doesn't fit in memory?

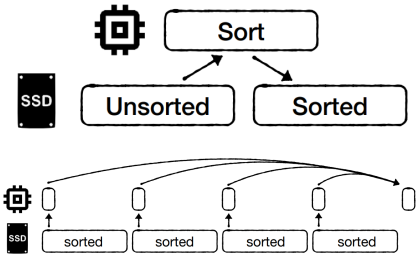


Setting up variables:

- M entries fit in memory
- N entries in total
- B entries per 4KB page
- $\Rightarrow M/B$ pages in mem, N/B pages to process in total

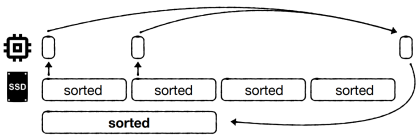
8.1 Multi-Way Merge Sort

For each chunk that fits into memory, sort them for N/M sorted temp. files for a cost of N/B I/Os (that is the partitioning phase)



Allocate a buffer for each input file and merge into output stream.

Too little memory requires multiple passes (here: 2 iters).



There are $\frac{N}{M}$ partitions, and we have $\frac{M}{B}$ buffers (pages in memory). **Each insertion merges $\frac{M}{B}$ partitions.** Total iterations:

$$\begin{aligned}\text{Iterations} &= \log_{M/B} N/M \\ &= \log_{\substack{\text{pages in} \\ \text{memory}}} \text{partitions} \\ &= \log_{\text{PG}_{\text{mem}}} \text{Partitions}\end{aligned}$$

Each iteration corresponds to how many levels of merging need to be done.

Each iteration does a full pass over data for a cost of $O(N/B)$

Merging costs $\frac{N}{B}\left\lceil\log_{\frac{M}{B}} N/M\right\rceil$ alongside N/B for partitioning.

Total calculated by

$$\underbrace{\frac{N}{B}}_{\text{partitoning}} + \underbrace{\frac{N}{B}\log_{\frac{M}{B}}\frac{N}{M}}_{\text{merging}} =$$

Total $O\left(\frac{N}{B}\log_{\frac{M}{B}}\frac{N}{B}\right)$.

Useful quantities:

- $\frac{N}{M}$: no. partitions
- $\frac{N}{B}$: total page count
- $\frac{M}{B}$: pages in memory
- M: entries that fit in memory
- N: entries in storage

Increasing memory count means merging more partitions per pass.

How much memory is required to merge **all** partitions in one pass? (I mean 2) – solve for M:

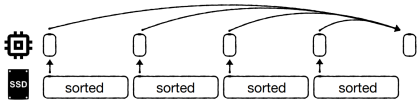
$$\log_{\frac{M}{B}}\frac{N}{B} = 2$$
$$\Rightarrow M = \sqrt{N \cdot B}$$

With this M, memory can accommodate $\sqrt{N/B}$ buffers.

With $M = \sqrt{N \cdot B}$ memory to partition data, this creates at most $N/M = \sqrt{N/B}$ sorted partitions. Then merging in one pass uses at most $\sqrt{N/B}$ input buffers.

For the cost of $O(N/B)$ I/Os overall.

The cost of this:



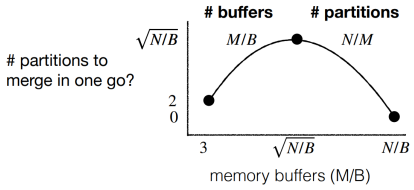
For all practical purposes, a 2-pass sorting-algorithm is practical.

8.2 Optimizing CPU cost

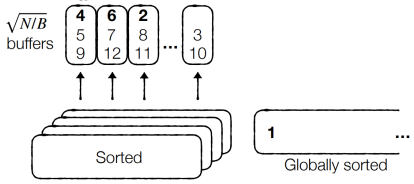
We expect $O(N\log_2(N))$.

Partitioning and sorting each chunk for a total of N/M chunks cost $O(N\log_2(M))$.

For merging:



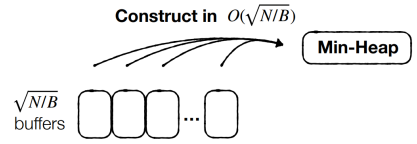
The idea is to traverse all $\frac{M}{B}$ buffers (e.g. $\sqrt{N/B}$ at the optimal) and pick minimum:



For a total of $O\left(\sqrt{N/B}\cdot N\right)$ comparisons.

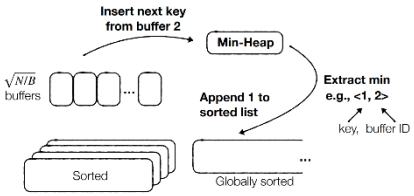
How do we sort this more efficiently?

For a min-heap with $\sqrt{N/B}$ items, one per buffer, we're able to **pop then insert** in log time.



So, for all buffers, the first elements of each buffer go in the min-heap. Each heap entry stores the buffer ID.

When item is extracted, insert next key from the heap entry's buffer ID.



Per entry, this costs $O\left(\log_2\sqrt{N/B}\right)$.

For N items that must be processed, this costs $O(N\log_2(N/B))$.

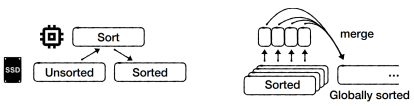
In all:

- **Partitioning** costs $O(N\log_2(M)) = O\left(N\lg\left(\sqrt{NB}\right)\right)$
- **Merging** costs $O\left(N\lg\left(\sqrt{N/B}\right)\right)$
- Total cost is $O(N\log_2 N)$

The same as in-memory algorithms.

Thus, the overall cost is:

$$\begin{cases} O(N\log_2(N)) & \text{CPU} \\ O\left(\frac{N}{B}\log_{\frac{M}{B}}\left(\frac{N}{B}\right)\right) & \text{I/O} \\ O\left(\frac{N}{B}\right) & M > \sqrt{MN} \end{cases}$$



8.3 Double-Buffering

Doubles the space required, 2 buffers required per partition, but eliminates the need to wait when an individual buffer runs out.

9 SQL Query Optimization

Firstly, statement trees should be the most furthest from balanced.

9.1 Selection

The index uses a B-tree.

For the simple `SELECT * FROM ... WHERE A = ...`

- Scans are just $O\left(\frac{N}{B}\right)$
- Unclustered indexes are $O(\log_B(N) + S)$
- Clustered indexes are $O\left(\log_B N + \frac{S}{B}\right)$

(Same table) When there are multiple selection conditions, like `SELECT * FROM ... WHERE X = "i" AND Y = "j"`

- Either unclustered (iterate cluster, check row)
  - Cost $\log_B(N) + |X_i|$ or $\log_B(N) + |Y_j|$, depending on which cluster is checked (ideally, should check the one with the smaller $|X_i|$ or $|Y_j|$
- Both unclustered
  - $2\log_B(N) + \frac{|X_i|}{B} + \frac{|Y_i|}{B} + |X_i \cap Y_j|$
- Composite index
  - $\log_B(N) + |X_i \cap Y_j|$

When taking the "either unclustered" approach, index through the table with less items.

Searching both indexes is better when $|X_i \cup Y_j|$ is small relative to $|X_i|$ and $|Y_j|$.

9.2 Order By

Any time there's an ORDER BY statement: unclustered index is helpful if result set is small, while a clustered index scan is good in all cases.

Quicksort, if result set fits in memory.

Heapsort if quicksort takes longer than expected, since heapsort has true $O(1)$ aux. space complexity.

9.3 Distinct

Sort and eliminate identical items, OR hash to identify identical items.

Sorting is better if you are going to need to sort the data anyway.

9.4 Group By

Index/sort by category, or hash by category.

9.5 Joining

Methods:

- Nested loop
  - Cost $\frac{|T_1||T_2|}{B}$
  - Strawman algorithm
- Block nested loop
  - Cost $\frac{|T_1||T_2|}{B^2}$
  - If one table fits into memory: $\frac{|T_1|}{B} + \frac{|T_2|}{B}$
- Index join
  - Use when indexes on the join keys
  - Either: $|T_1| \log_B |T_2|$
  - Both: $|T_1| + |T_2|$
- Sort-merge join
  - Use when both relations much larger than memory
  - I/O cost $\frac{|T_1|}{B} + \frac{|T_2|}{B}$
  - CPU cost $|T_1| \lg |T_2| + |T_2| \lg |T_1|$
  - Mem cost $\sqrt{\max(|T_1|, |T_2|)\, B}$
- Grace hash join
  - I/O cost $\frac{|T_1|}{B} + \frac{|T_2|}{B}$
  - CPU cost $|T_1| + |T_2|$
  - Memory cost $\sqrt{\min\begin{pmatrix}|T_1|,\\ |T_2|\end{pmatrix} B}$

9.6 Cardinality Estimation

Estimate X_i as $\frac{N}{|X|} = \frac{\text{no. rows}}{\text{unique } X \text{ values}}$

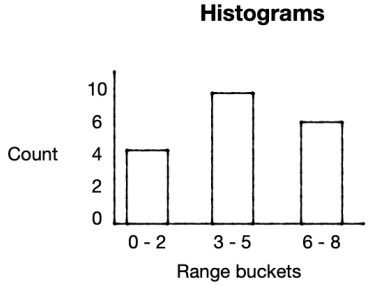
The problem: we don't know N, since on insertion we do not know if the value is unique unless we have an index

You can put a cardinality counter for each index

If no index:

Periodically scan and count no. of unique entries

We can instead use histograms, with range buckets. Estimate $|X_i|$ as $\frac{\text{bucket count}}{\text{bucket range}}$



9.7 Query Optimization

TBA

10 Column Stores

Great for large statistical calculations and batch updates. Updating one at a time is way too expensive. Also, items should still be fixed length.

Scanning a column in memory would require hashing overheads for each page. Therefore, just read and map at column granularity.

Basic model: a store contains `A[], B[], C[]`, and so on. Different columns are not contiguous to each other.

10.1 Processing Queries

In a column store: `SELECT min(C) FROM Table WHERE A>10 AND B<20`

Approaches:

- Early materialization (the dumb way): For each item: Check A – if pass, then B – if pass then deal with C
  - Unfortunately, more random I/Os and more CPU branch mispredictions, and incompatible with SIMD
- Late materialization: process one column at a time
  - Scan column A fetch qualifying IDs, and then start scanning B
  - Works best when A has the fewest qualifiers (best to filter on more selective columns first)
  - Requires more memory to withhold intermediate results, but entails better cache behavior and CPU efficiency
  - For intermediate results:
    - Array of integer IDs work best if few results qualify
    - Bitmap good when many results qualify

Ways to optimize:

- Use zone maps (each partition stores min and max)
- Store columns sequentially to allow pre-fetching and I/O parallelism

10.2 Handling Insertions

Operation: `INSERT into TABLE VALUES (a,b,c)`, approaches:

- **In-place**: directly insert to the end of each column by reading/writing to storage. Cost: O(no. cols)
- **In-memory buffering:** I think you get the idea

10.3 Handling Deletes

There couldn't be a better way.

- Create holes.
  - Now you need an additional bit per column to track if an entry has been deleted
- Employ delete column
  - Additional column signals entry is deleted and this is eventually merged in. Anything that is stored outside that helps indicate the entry is deleted works.

10.4 Handling Updates

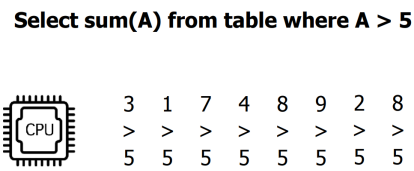
For query `UPDATE TABLE SET B="b1", C="c1" WHERE A="x"`

Delete and re-insert. Store the deletion timestamp for each delete operation and store the insertion timestamp for each insert.

There is no better way. This is why modifications are best done in batch and offline.

10.5 SIMD

SIMD operations can apply one instruction in parallel to multiple values within one cache line.



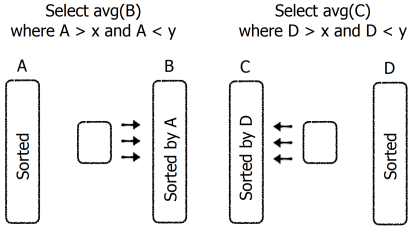
10.6 Compression

Options:

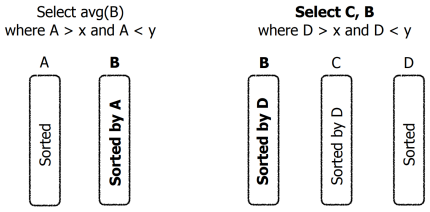
- Bit-vector encoding: Employ one bit string for each possible value indicating if the entry has the given value (this effectively makes each entry an Enum)
- Dictionary encoding: Employ a dictionary with smaller strings to represent larger ones. Slower as we need to read more bits
- Run-length encoding
  - `Cat, Dog, Cat*4, Horse, Cat*2, Dog`
  - Compatible with dictionary encoding
  - Lose out on index access

10.7 Indexing In Column Stores

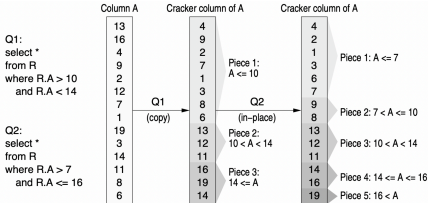
It's still useful to index columns to make it easy to find where something is. However, that still isn't good enough. Our best approach is to **sort the columns.**



Unfortunately, this might result in columns being duplicated.



We solve this problem by employing database cracking:



For Q1: partitions the table into $A \leq 10$, $10 < A < 14$, $14 \leq A$. Cracking up to what's necessary. Future operations can crack harder.

- Queries adaptively refine the sorting and speed up subsequent queries
- We can begin querying the data immediately without creating too many projections from the onset, which would take a while

11 Runtime Summary

If internal nodes are not in mem, B-tree Q&IR$\times \log_B(N)$, LSM Q&IR: $\times \log_T\left(\frac{N}{B}\right)$. LSM trees assumed they were clustered.

$$L = \log_T\left(\frac{N}{P}\right)$$

$	Apd. only	Sorted list	B-tree
Q	N/B	$\log\left(\frac{N}{B}\right)$	1
IR	0	N/B	1
IW	$1/B$	N/B	$1+GC$
S	N/B	$\log\left(\frac{N}{B}\right)+\frac{S}{B}$	S Clust: S/B
M	B		N/B

$	LSM-BSC	LSM-TRD	LSM-LVL
Q	$\lg\left(\frac{N}{P}\right)$	LT	L
IR	$\frac{\lg\left(\frac{N}{P}\right)}{B}$	$\frac{L}{B}$	$\frac{LT}{B}$
IW	$\frac{\lg\left(\frac{N}{P}\right)}{B}$	$\frac{L}{B}$	$\frac{LT}{B}$
S	$Q+\frac{S}{B}$	$Q+\frac{S}{B}$	$Q+\frac{S}{B}$
M	N/B	N/B	N/B

$	LSM-LVL	LSM-BLM	MKY+DV
Q	L	$2^{-M}L$	2^{-M}
IR	$\frac{LT}{B}$	$\leftarrow$	$\frac{L+T}{B}$
IW	$\frac{LT}{B}$	$\leftarrow$	$\frac{L+T}{B}$
S	$Q+\frac{S}{B}$	$Q+\frac{S}{B}$	$Q+\frac{S}{B}$
M	N/B	N/B	N/B

11.1 Bloom Filters

For bloom filters, CPU cost is:

- Insertions $= M\ln(2) = k$, $O(M)$
- Positive query $= M\ln(2)$, $O(M)$
- Avg. Negative query $= 2$
- FPR $= 2^{-M\ln(2)}$

11.2 External Sorting

External sorting:

- $M \geq \sqrt{N \cdot B}$ for one-pass
- If so: $O\left(\frac{N}{B}\right)$ I/O R&W, $O\left(N\lg(N)\right)$ CPU
- For non-one pass it costs $O\left(\frac{N}{B}\log_{\frac{M}{B}}\left(\frac{N}{B}\right)\right)$

SSD write amplification:

SSD WA approximation: where x is % valid pages. WA is at its worst when all erase units have the same number of invalid pages (which means finding the erase unit with the least no. of pages that need to be copied is not going to be so optimal):

$$1+\left(\frac{x}{1-x}\right)$$

Cost model assuming uniformly randomly distributed writes:

$$1+\frac{1}{2}\cdot\frac{L/P}{1-L/P}$$

Where:

$$\frac{L}{P} = \frac{\text{valid}}{\text{empty}+\text{valid}+\text{invalid}}$$

11.3 Log Rules

Log rules:

$$\log_B N = D$$
$$\Rightarrow b^D = N$$
$$\log_a b = \frac{\log_c b}{\log_c a}$$
$$\log_a 1 = 0$$
$$\log_a a = 1$$

11.4 Common Calculations

Common calculations:

$$2^0 = 1$$
$$2^1 = 2$$
$$2^2 = 4$$
$$2^3 = 8$$
$$2^4 = 16$$
$$2^5 = 32$$
$$2^6 = 64$$
$$2^7 = 128$$
$$2^8 = 256$$
$$2^9 = 512$$
$$2^{10} = 1024$$
$$2^{11} = 2048$$
$$2^{12} = 4096$$

$$2^{10} = 1024 = 1\text{KB}$$
$$2^{20} = 1048576 = 1\text{MB}$$
$$2^{30} = 1073741824 = 1\text{GB}$$
$$2^{40} = 1\text{TB}$$

$$1^2 = 2$$
$$10^2 = 100$$
$$100^2 = 10000 = 10\text{K}$$
$$1000^2 = 1000000 = 1\text{M}$$

11.5 Page Read / Write Speed

Accessing sequential pages on the disk is cheap since the seeker does not need to move.

- A disk I/O takes 10ms to read 4KB pages.
  - Throughput: 0.4 MB/s
- An SSD I/O takes 100μs to read 4KB pages.
  - Throughput: 40 MB/s